



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251176
Nama Lengkap	David Alexsi Cahyadi
Minggu ke / Materi	09 / Pengolahan String dan Regular Expression

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Pengantar String

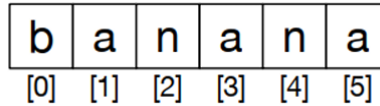
String adalah urutan karakter-karakter yang menjadi satu kesatuan dan digunakan dalam program komputer untuk menyimpan kalimat, baik panjang maupun pendek. String merupakan tipe data yang mampu menyimpan huruf atau karakter, dan secara internal disimpan dalam kode ASCII.

Perlu diketahui bahwa tidak semua bahasa pemrograman memiliki tipe data String secara bawaan, seperti misalnya bahasa C. String bukan termasuk tipe data dasar, karena pada dasarnya ia menyimpan lebih dari satu nilai tunggal sebagai satu kesatuan. Beberapa bahasa pemrograman bahkan menyebut bahwa String pada dasarnya adalah kumpulan tipe data karakter, atau yang dikenal sebagai *array of character* maupun *list of character*.

Pengaksesan String dan Manipulasi String

String dapat dibuat dengan cara yang sederhana, yaitu menggunakan deklarasi variabel yang langsung diisi dengan data berupa teks. String dapat diakses dengan satu kesatuan dengan menyebut nama variabelnya, atau per huruf dengan menyebutkan indeksinya. Indeks pada string dimulai dari angka 0, seperti layaknya pada list, dan indeks tersebut haruslah berupa bilangan bulat, bukan pecahan.

Pada memori komputer, string disimpan secara urut menggunakan struktur seperti list yang berisi huruf-huruf dengan indeks yang dimulai dari nol. Sebagai ilustrasi, string “banana” akan disimpan dengan huruf ‘b’ pada indeks [0], ‘a’ pada [1], ‘n’ pada [2], dan seterusnya hingga indeks [5].



Gambar 8.1: Bentuk string di dalam memory, Sumber: <https://shorturl.at/Vvatx>

Selain itu, dua buah string dapat digabungkan menggunakan operator **+**, seperti pada contoh **“Matahari” + ‘Bakti’** yang menghasilkan **“MatahariBakti”**.

```
1  namasaya = "Antonius Rachmat C"
2  temansaya1 = "Yuan Lukito"
3  temansaya2 = 'Laurentius Kuncoro'
4  temansaya3 = "Matahari" + 'Bakti'
5
6  print(temansaya3)
7  print(namasaya[0]) #'A'
8  print(namasaya[9]) #'R'
9  print(temansaya1[1]) #'u'
10
11  huruf = temansaya2[0]
12  print(huruf) #'L'
```

Gambar 8.2: Contoh penggunaan operator **+** pada string, Sumber: <https://shorturl.at/Vvatx>

Operator dan Metode String

Operator in

Pada string, kita dapat memeriksa apakah suatu kata atau kalimat merupakan bagian (*substring*) dari kalimat menggunakan operator **in**. Hasil dari operasi ini berupa nilai boolean **True** or **False**. Misalnya, jika memeriksa apakah kata **“saya”** ada di dalam kalimat **“saya mau makan”**, hasilnya adalah **True**. Sebaliknya, jika kata yang dicari tidak ada dalam kalimat, hasilnya adalah **False**.

Selain **in**, pada string juga dapat dilakukan operasi perbandingan (*comparison*) menggunakan operator seperti **>**, **<**, atau **==**, yang juga menghasilkan **True** atau **False**.

Fungsi len

Untuk mengetahui panjang sebuah string (jumlah karakternya), digunakan fungsi **len()**. Karena indeks dimulai dari 0, maka huruf terakhir berada pada indeks **len(string) – 1**. Sebagai alternatif yang lebih praktis, Python juga mendukung penggunaan indeks negatif, di mana **-1** merujuk pada karakter terakhir, **-2** pada karakter kedua dari belakang, dan seterusnya.

Traversing String

Untuk menampilkan string huruf demi huruf, dapat digunakan dua pendekatan. Pertama, dengan menggunakan loop **while** yang mengakses setiap karakter berdasarkan indeksinya secara manual. Kedua, dengan menggunakan loop **for** yang secara otomatis mengiterasi setiap karakter dalam string tanpa perlu mengelola indeks secara eksplisit.

String Slice

String slice adalah teknik untuk menampilkan sebagian dari sebuah string (*substring*) dengan menggunakan sintaks **string[awal:akhir]**. Indeks awal bersifat inklusif sedangkan indeks akhir bersifat eksklusif (akhir-1). Bagian awal atau akhir boleh dikosongkan, jika awal dikosongkan maka dimulai dari indeks 0, dan jika akhir dikosongkan maka akan diambil hingga karakter terakhir.

Penting untuk dipahami bahwa String bersifat **immutable**, artinya data di dalam string tidak dapat diubah secara langsung saat program berjalan. Jika ingin melakukan modifikasi, hasilnya harus disimpan ke dalam variabel baru.

Metode-Metode String

Python menyediakan fungsi berbagai metode bawaan untuk memanipulasi string. Beberapa yang sering digunakan antara lain: **capitalize()** untuk mengubah huruf pertama menjadi huruf besar, **count()** untuk menghitung jumlah kemunculan suatu substring, **strip()** untuk menjadi beberapa bagian berdasarkan pemisah tertentu. Perlu diingat bahwa semua metode yang mengembalikan string akan menghasilkan string baru, bukan mengubah string aslinya, karena sifat immutable tersebut.

Traversing String

Untuk menampilkan string huruf demi huruf, dapat digunakan dua pendekatan. Pertama, dengan menggunakan loop **while** yang mengakses setiap karakter berdasarkan indeksinya secara manual. Kedua, dengan menggunakan loop **for** yang secara otomatis mengiterasi setiap karakter dalam string tanpa perlu mengelola indeks secara eksplisit.

Operator * dan + pada String

Operator **+** dapat digunakan untuk menggabungkan dua buah string (*concatenation*), sedangkan operator ***** dapat digunakan untuk mengulang tampilan sebuah string sebanyak kelipatan yang ditentukan. Misalnya, **“ulang” * 4** akan menghasilkan output **“ulangulangulangulang”**.

Parsing String

Parsing string adalah teknik menelusuri string bagian demi bagian untuk mendapatkan, menemukan, atau mengubah bagian string tertentu yang diinginkan. Teknik ini umumnya memanfaatkan kombinasi metode **split()** dan pengaksesan indeks.

Sebagai contoh, misal terdapat kalimat **“Saudara-saudara, pada tanggal 17-08-1945 Indonesia merdeka”** dan kita ingin mengambil tanggal dari kalimat tersebut lalu

menyusunnya ulang menjadi format **08/17/1945**. Langkah yang dilakukan adalah pertama memecah kalimat berdasarkan spasi sehingga terbentuk token-token kata. Selanjutnya, dicari token yang diawali dengan angka menggunakan metode **isdigit()**. Setelah token tanggal ditemukan, token tersebut dipecah lagi berdasarkan karakter -, lalu disusun ulang sesuai format yang diinginkan.

Pengantar Regex

Dari pengalaman mengolah string dengan teknik biasa, dapat dirasakan bahwa pendekatan standar seperti **split()** dan **find()** terkadang kurang cukup untuk menangani pola-pola string yang kompleks. Untuk itulah hadir *regular expression (regex)* sebagai solusi yang lebih mudah dan lebih kuat (powerful).

Regular expression adalah ekspresi pola yang dari kumpulan karakter khusus, yang digunakan untuk menemukan, mencocokkan, mengganti, atau menghapus bagian string yang sesuai dengan pola tersebut. Python mendukung penggunaan regex melalui library bawaan **re**, yang dapat digunakan dengan menambahkan **import re** di awal program.

Salah satu fungsi paling dasar dari library **re** adalah **re.search()**, yang digunakan untuk memeriksa apakah suatu pola ditemukan dalam sebuah string. Selain itu, terdapat karakter khusus **^** yang digunakan untuk menandai bahwa pola harus ditemukan di awal baris. Misalnya, **re.search('^From: ', line)** hanya akan mencocokkan baris yang benar-benar diawali dengan kata **From:** .

Meta Character, Escaped Character, Set of Character, dan Fungsi Regex pada Library Python

Agar dapat menggunakan regex secara efektif, perlu dipahami terlebih dahulu berbagai karakter khusus (*special characters*) beserta fungsinya. Karakter `[ ]` digunakan untuk mendefinisikan himpunan karakter, titik `.` mewakili karakter apapun kecuali newline, `^` menandai awalan string, `$` menandai akhiran string, `*` berarti nol hingga tak terhingga karakter, `+` berarti minimal satu karakter, `?` berarti karakter bersifat opsional, dan `|` berarti “atau” antara dua pola.

Selain itu, terdapat *escaped characters* yang memiliki makna khusus dengan regex. Di antaranya adalah `\d` untuk mencocokkan digit (angka 0-9), `\D` untuk karakter yang bukan digit, `\s` untuk karakter whitespace (spasi, tab, enter), `\S` untuk karakter yang bukan whitespace, `\w` untuk karakter word (huruf, angka, dan underscore), serta `\W` untuk karakter yang bukan word.

Untuk mendefinisikan himpunan karakter secara spesifik digunakan tanda kurung siku `[ ]`. Misalnya, `[a-zA-Z]` berarti satu karakter huruf kecil atau huruf besar, `[0-9]` berarti satu digit angka, dan `[^bmx]` berarti satu karakter yang bukan ‘b’, ‘m’, maupun ‘x’.

Terakhir, Python menyediakan empat fungsi utama dalam library **re** untuk bekerja dengan regex. Fungsi **findall()** mengembalikan semua bagian string yang cocok dengan pola dalam bentuk list. Fungsi **search()** mengembalikan hasil pencocokan pertama yang ditemukan. Fungsi **split()** memecah string berdasarkan pola regex sebagai pemisah. Dan fungsi **sub()** digunakan untuk mengganti bagian string yang cocok dengan pola menjadi string pengganti yang diinginkan.

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

Github: https://github.com/DavidCahyadi1013/71251176_David-Alexsi-Cahyadi.git

Latihan 8.1

Soal:

Latihan 8.1 Buatlah sebuah program yang dapat mendeteksi apakah suatu kata adalah anagram dari kata lainnya atau bukan! Anagram adalah kata yang dibolak-balik susunan hurufnya sama. Misal: mata anagram dengan atma, maat, taam, tama, dsb

Source Code:

```
k1 = input("Masukkan kata pertama: ")
k2 = input("Masukkan kata kedua: ")

k1 = k1.lower()
k2 = k2.lower()

urut1 = sorted(k1)
urut2 = sorted(k2)

if urut1 == urut2:
    print (f"\n'{k1}' dan '{k2}' ADALAH anagram!")
else:
    print (f"\n'{k1}' dan '{k2}' BUKAN anagram!")
```

Output:

```
PS D:\PrAlpro\Pertemuan 9> & C:/Users/dvdal/AppData/Local/Programs/Python/Python313/python.exe "d:/PrAlpro/Pertemuan 9/soal01.py"
Masukkan kata pertama: guga
Masukkan kata kedua: uagg
'guga' dan 'uagg' ADALAH anagram!
PS D:\PrAlpro\Pertemuan 9> & C:/Users/dvdal/AppData/Local/Programs/Python/Python313/python.exe "d:/PrAlpro/Pertemuan 9/soal01.py"
Masukkan kata pertama: mata
Masukkan kata kedua: taam
'mata' dan 'taam' ADALAH anagram!
```


Penjelasan:

Program dimulai dengan meminta pengguna memasukkan dua kata melalui fungsi **input()**, yang disimpan ke dalam variabel **k1** dan **k2**. Setelah kedua kata didapat, keduanya diubah menjadi huruf kecil semua menggunakan metode **.lower()** agar program tidak membedakan huruf besar dan kecil, sehingga kata seperti **“Mata”** dan **“mata”** tetap dianggap sama.

Selanjutnya, huruf-huruf dari masing-masing kata diurutkan menggunakan fungsi **sorted()**. Fungsi ini akan memecah string menjadi list huruf kemudian mengurutkannya secara abjad/alfabet. Hasil pengurutan disimpan ke variabel baru **urut1** dan **urut2**, karena string bersifat **immutable** sehingga tidak bisa langsung diubah.

Terakhir, program membandingkan **urut1** dan **urut2** menggunakan operator **==**. Jika kedua hasil pengurutan identik, maka dapat dihasilkan kedua kata memiliki huruf yang sama persis hanya berbeda susunannya, sehingga program mencetak bahwa kedua kata **adalah anagram**. Sebaliknya jika berbeda, program mencetak bahwa keduanya **bukan anagram**.

Latihan 8.2

Soal:

Latihan 8.2 Buatlah suatu program yang dapat menghitung frekuensi kemunculan suatu kata yang ada pada String. Misal terdapat kalimat "Saya mau makan. Makan itu wajib. Mau siang atau malam saya wajib makan". Ditanyakan kata "makan". Output: makan ada 3 buah

Source Code:

```
kalimat = input("Masukkan kalimat: ")
kata = input("Masukkan kata yang dicari: ")

kalimat = kalimat.lower()
kata = kata.lower()
```

```
kalimat = kalimat.replace(".", "")

pecah = kalimat.split()
jumlah = pecah.count(kata)
print(f"\n{kata} ada {jumlah} buah")
```

Output:

```
PS D:\PrAlpro\Pertemuan 9> & C:/Users/dvdal/AppData/Local/Programs/Python/Python313/python.exe "d:/PrAlpro/Pertemuan 9/soal02.py"
Masukkan kalimat: Saya mau makan. Makan itu wajib. Mau siang atau malam saya wajib makan
Masukkan kata yang dicari: makan

makan ada 3 buah
PS D:\PrAlpro\Pertemuan 9> & C:/Users/dvdal/AppData/Local/Programs/Python/Python313/python.exe "d:/PrAlpro/Pertemuan 9/soal02.py"
Masukkan kalimat: koh wisnu sukanya lari sama koh filip, lalu koh wisnu dan koh filip bertemu mas karel lagi sibuk joki strava
Masukkan kata yang dicari: koh

koh ada 4 buah
```

Penjelasan:

Program dimulai dengan meminta pengguna memasukkan sebuah kalimat dan kata yang ingin dicari melalui fungsi **input()**, yang masing-masing disimpan ke dalam variabel **kalimat** dan **kata**. Setelah kedua input didapat, keduanya diubah menjadi huruf kecil semuanya menggunakan metode **.lower()** agar pencarian tidak membedakan huruf besar dan kecil, sehingga kata seperti **“Makan”** dan **“makan”** tetap dianggap sama.

Selanjutnya, tanda titik **“.”** yang menempel pada akhir kata dibersihkan menggunakan metode **.replace(“.”, “”)**. Hal ini penting agar kata seperti **“makanan.”** tidak dianggap berbeda dengan **“makan”** saat proses penghitungan berlangsung.

Setelah kalimat bersih dari tanda baca, kalimat dipecah menjadi token-token kata menggunakan metode **.split()**. Hasilnya berupa list yang berisi setiap kata secara terpisah dan disimpan ke variabel **pecah**. Kemudian metode **.count()** dipanggil pada list **pecah** untuk menghitung berapa kali kata yang dicari muncul di dalamnya. Hasil hitungan disimpan ke variabel **jumlah** dan ditampilkan ke layar.

Latihan 8.3

Soal:

Latihan 8.3 Buatlah suatu program yang dapat menghapus semua spasi yang berlebih pada sebuah string, dan menjadikannya satu spasi normal! Misal: "saya tidak suka memancing ikan " Output: "saya tidak suka memancing ikan"

Source Code:

```
k = input ("Masukkan kalimat : ")
hasil = ' '.join(k.split())
print(hasil)
```

Output:

```
PS D:\PrAlpro\Pertemuan 9> & C:/Users/dvdal/AppData/Local/Programs/Python/Python313/python.exe "d:/PrAlpro/Pertemuan 9/soal03.py"
Masukkan kalimat : mas philip      kayang dilihat oleh prop weesnu
mas philip kayang dilihat oleh prop weesnu
PS D:\PrAlpro\Pertemuan 9> & C:/Users/dvdal/AppData/Local/Programs/Python/Python313/python.exe "d:/PrAlpro/Pertemuan 9/soal03.py"
Masukkan kalimat : kicau      kicau      mania
kicau kicau mania
```

Penjelasan:

Program dimulai dengan meminta pengguna memasukkan sebuah kalimat melalui fungsi **input()** yang disimpan ke dalam variabel **k**.

Selanjutnya, **k.split()** dipanggil tanpa argumen untuk memecah kalimat menjadi token-token kata. Ketika **.split()** dipanggil tanpa argumen, Python secara otomatis mengabaikan semua spasi berlebih antara kata sehingga hasilnya hanya berupa list kata-kata bersih tanpa spasi kosong.

Kemudian ‘**.join()**’ menggabungkan kembali semua kata dalam list tersebut dengan tepat satu spasi sebagai pemisah antar katanya. Hasil penggabungan disimpan ke variabel **hasil** dan ditampilkan ke layar.

Latihan 8.4

Soal:

Latihan 8.4 Buatlah suatu program mengetahui kata terpendek dan terpanjang dari suatu kalimat yang diinputkan! Misal: "red snakes and a black frog in the pool" Output: terpendek: a, terpanjang: snakes

Source Code:

```
k = input ("Masukkan kalimat : ")
k= k.lower()
pecah = k.split()

pendek = pecah[0]
panjang = pecah [0]

for i in pecah:
    if len (i) < len (pendek):
        pendek = i
    if len(i) > len (panjang):
        panjang = i
print(f"terpendek: {pendek}")
print(f"terpanjang: {panjang}")
```

Output:

```
PS D:\PrAlpro\Pertemuan 9> & C:/Users/dvdal/AppData/Local/Programs/Python/Python313/python.exe "d:/PrAlpro/Pertemuan 9/soal04.py"
Masukkan kalimat : red snakes and a black frog in the pool
terpendek: a
terpanjang: snakes
PS D:\PrAlpro\Pertemuan 9> & C:/Users/dvdal/AppData/Local/Programs/Python/Python313/python.exe "d:/PrAlpro/Pertemuan 9/soal04.py"
Masukkan kalimat : ki kicaw maniakkss kicaw mania
terpendek: ki
terpanjang: maniakkss
```

Penjelasan:

Program dimulai dengan meminta pengguna memasukkan sebuah kalimat melalui fungsi **input()**. Kalimat kemudian diubah ke huruf kecil menggunakan **.lower()** lalu dipecah menjadi list kata-kata menggunakan **.split()** dan disimpan ke variabel **pecah**.

Selanjutnya, variabel **pendek** dan **panjang** disiapkan dengan nilai-nilai awal berupa kata pertama dari list yaitu **pecah[0]** sebagai pembanding awal. Program kemudian melakukan perulangan pada setiap kata dalam list. Pada setiap iterasi, panjang kata diperiksa menggunakan fungsi **len()**. Jika panjang kata lebih pendek dari **pendek** maka variabel **pendek** diperbarui, dan jika lebih panjang dari **panjang** maka variabel **panjang** diperbarui.

Latihan 8.5

Soal:

Latihan 8.5 Anda diminta untuk mencari seluruh teks yang berupa tanggal dengan format YYYY-MM-DD dan kemudian seluruh tanggal tersebut diambil dan ditampilkan kembali dalam format DD-MM-YYYY ditambah dengan perhitungan selisih dengan tanggal sekarang dalam hari.

Contoh:

Pada tanggal 1945-08-17 Indonesia merdeka. Indonesia memiliki beberapa pahlawan nasional, seperti Pangeran Diponegoro (TL: 1785-11-11), Pattimura (TL: 1783-06-08) dan Ki Hajar Dewantara (1889-05-02).

Hasil:

1945-08-17 00:00:00 selisih 27209 hari

1785-11-11 00:00:00 selisih 85561 hari

1783-06-08 00:00:00 selisih 86448 hari

1889-05-02 00:00:00 selisih 47769 hari

Source Code:

```
import re
from datetime import datetime

t = input("Masukkan teks: ")
tgl = re.findall(r'\d{4}-\d{2}-\d{2}', t)
```

```

sekarang = datetime.now()
for i in tgl:
    tg = datetime.strptime(i, '%Y-%m-%d')
    selisih = abs((sekarang - tg).days)
    print(f"{tg} selisih {selisih} hari")

```

Output:

```

PS D:\PrAlpro\Pertemuan 9> & C:/Users/dvdal/AppData/Local/Programs/Python/Python313/python.exe "d:/PrAlpro/Pertemuan 9/soal05.py"
Masukkan teks: Pada tanggal 1945-08-17 Indonesia merdeka. Indonesia memiliki beberapa pahlawan nasional, seperti Pangeran Diponegoro (TL: 1785-11-11), Pattimura (TL: 17
83-06-08) dan Ki Hajar Dewantara (1889-05-02).
1945-08-17 00:00:00 selisih 29464 hari
1785-11-11 00:00:00 selisih 87816 hari
1783-06-08 00:00:00 selisih 88703 hari
1889-05-02 00:00:00 selisih 50024 hari

```

Percobaan di uji pada tanggal (18/04/2026)

Penjelasan:

Program dimulai dengan mengimport **re** untuk kebutuhan regex dan **datetime** dari library **datetime** untuk kebutuhan perhitungan tanggal. Pengguna kemudian diminta memasukkan teks bebas melalui **input()** yang disimpan ke variabel **t**.

Selanjutnya, **re.findall()** digunakan dengan pola **\d{4}-\d{2}-\d{2}** untuk mencari semua tanggal berformat **YYYY-MM-DD** di dalam teks. Pola ini berarti tepat 4 digit angka, diikuti -, lalu tepat 2 digit angka, diikuti -, lalu tepat 2 digit angka. Semua tanggal yang ditemukan dikembalikan dalam bentuk list dan disimpan ke variabel **tgl**.

Kemudian **datetime.now()** dipanggil untuk mendapatkan tanggal hari ini dan disimpan ke variabel **sekarang**. Program lalu melakukan perulangan pada setiap tanggal yang ada di dalam list **tgl**. Pada setiap iterasi, tanggal yang masih berupa string dikonversi menjadi objek **datetime** menggunakan **strptime()** dengan format **'%Y-%m-%d'** dan disimpan ke variabel **tg**. Selisih antara tanggal sekarang dan tanggal tersebut dihitung dengan operator **-**, kemudian diambil nilai harinya menggunakan **.days**. Fungsi **abs()** digunakan agar hasil selisih selalu bernilai positif. Terakhir, hasil ditampilkan ke layar.

Latihan 8.6

Soal:

Latihan 8.6 Anda diminta untuk mencari seluruh teks yang berupa email dan kemudian ambil semua username dari email tersebut untuk digenerate password random 8 karakter yang terdiri dari angka dan huruf.

Contoh:

Berikut adalah daftar email dan nama pengguna dari mailing list:

anton@mail.com dimiliki oleh antonius

budi@gmail.co.id dimiliki oleh budi anwari

slamet@getnada.com dimiliki oleh slamet slumut

matahari@tokopedia.com dimiliki oleh toko matahari

Hasil:

anton@mail.com username: anton , password: 8u78A2UD

budi@gmail.co.id username: budi , password: bdP066Ld

slamet@getnada.com username: slamet , password: Ab1FiHXb

matahari@tokopedia.com username: matahari , password: 5KYyaP6

Source Code:

```
import re
import random
import string

print("Berikut adalah daftar email dan nama pengguna dari mailing list: ")

lines = []
while True:
    baris = input()
    if baris == "":
        break
    lines.append(baris)
```

```

t= "\n".join(lines)
email = re.findall(r'[a-zA-Z0-9._]+@[a-zA-Z0-9]+(?:\.[a-zA-Z]+)+', t)

def pw():
    ch= string.ascii_letters + string.digits
    return ''.join(random.choice(ch) for _ in range(8))

for i in email:
    nama = i.split('@')[0]
    sandi= pw()
    print(f"{i} username: {nama}, password: {sandi}")

```

Output:

```

PS D:\PrAlpro\Pertemuan 9> & C:/Users/dvdal/AppData/Local/Programs/Python/Python313/python.exe "d:/PrAlpro/Pertemuan 9/soal06.py"
Berikut adalah daftar email dan nama pengguna dari mailing list:
anton@mail.com dimiliki oleh antonius
budi@mail.co.id dimiliki oleh budi anwari
slamet@getnada.com dimiliki oleh slamet slumut
matahari@tokopedia.com dimiliki oleh toko matahari

anton@mail.com username: anton, password: BpP2PhRI
budi@mail.co.id username: budi, password: 7f6Cjw0C
slamet@getnada.com username: slamet, password: 0wtNd4eE
matahari@tokopedia.com username: matahari, password: V1xPchzc

```

Penjelasan:

Program dimulai dengan mengimport tiga library yaitu **re** untuk regex, **random** dan **string** untuk kebutuhan generate password. Program kemudian mencetak kalimat pembuka, lalu meminta pengguna memasukkan teks secara baris per baris menggunakan perulangan **while True**. Setiap baris yang diinput disimpan ke dalam list **lines**. Perulangan berhenti ketika pengguna menekan Enter tanpa mengetik apapun (input kosong). Seluruh baris kemudian digabungkan menjadi satu teks utuh menggunakan “**\n**”.**join(lines)** dan disimpan ke variabel **t**.

Selanjutnya, **re.findall()** digunakan dengan pola **[a-zA-Z0-9._]+@[a-zA-Z0-9]+(?:\.[a-zA-Z]+)+** untuk mencari semua alamat email yang valid di dalam teks. Semua email yang ditemukan dikembalikan dalam bentuk list dan disimpan ke variabel **email**.

Sebuah fungsi **pw()** didefinisikan untuk membuat password acak 8 karakter. Fungsi ini menggabungkan **string.ascii_letters** dan **string.digits** sebagai kumpulan karakter yang tersedia, kemudian memilih 8 karakter secara acak menggunakan **random.choice()** dan menggabungkannya menggunakan **"".join()**.

Pada tahap akhir, program melakukan perulangan pada setiap email yang ditentukan. Username diambil dengan memecah email menggunakan **.split('@')** dan mengambil bagian indeks **[0]** yaitu bagian sebelum tanda **@**. Fungsi **pw()** dipanggil untuk menghasilkan password acak, dan keduanya ditampilkan bersama emailnya.